Department of Computer Science & Engineering

Software Engineering

UE24CS341A

Lab 7: Fuzz Testing Submission

Name: Devansh Gupta	SRN: PES1UG24CS145	Section: C

Section 1

Section 1 - Objective and setup

Task: Use Hypothesis to discover input validation bugs in the assigned parking application, fix them, and verify the fixes.

Assigned repository: https://github.com/SETAPESU26/L6_L7_19_parking_fee (Lab 7/). Source commit: fef20d42191cdd992b14c5717db612b929c2cfd0.

The repository filenames parking_fee.py and test_parking_fee.py correspond to processor.py and test_processor.py in the handout.

Environment used: 3.14.6 (main, Aug 10 2026, 06:36:12) [GCC 16.1.1 20260728]. Dependencies: pytest 9.2.0 and Hypothesis 6.170.15.

Commands: python -m pip install -r requirements.txt; python parking_fee.py; python -m pytest test_parking_fee.py.

Method: Record the original application output, complete the three starter properties, retain their failures, fix each helper, then rerun both those properties and the expanded suite.

The final suite has 79 helper test instances (14 Hypothesis property instances and 65 explicit regression cases), plus nine CLI cases: 88 passing tests in total. A property instance may execute hundreds of generated examples; the pytest test count is not the generated-input count.

Arbitrary Unicode is useful for finding crashes, but rarely resembles a valid time range or plate. Structured strategies also generate valid values and check their exact outputs against independent expectations. Hypothesis shrinking reduces a failure to a small example for diagnosis.

Name: Devansh Gupta	SRN: PES1UG24CS145	Section: C

Section 2

Section 2 - Step 2: Run the original application

Input: 09:30-11:45, car, ka01ab1234. The original program prints 135 minutes, rate 30, and KA 01 AB 1234.

An empty time input causes ValueError and stops the program before the remaining prompts. The baseline application is preserved unchanged in baseline/parking_fee.py.

Screenshot 1: Original application, valid input

Lab 7 · Before · application_valid

Browser screenshot of saved command output. Output below is reproduced verbatim.

```
$ python -u parking_fee.py
Working directory: Lab-7/baseline
Exit code: 0
Input supplied (JSON escaped): "09:30-11:45\ncar\nka01ab1234\n"
```

```
==== Running minutes_parked ====
Enter entry and exit time (e.g. 09:30-11:45): Minutes Parked: 135

==== Running hourly_rate ====
Enter vehicle type (bike/car/truck): Hourly Rate: 30

==== Running format_plate ====
Enter a number plate (e.g. ka01ab1234): Formatted Plate: KA 01 AB 1234
```

Screenshot 2: Original application, invalid input

Lab 7 · Before · application_invalid

Browser screenshot of saved command output. Output below is reproduced verbatim.

```
$ python -u parking_fee.py
Working directory: Lab-7/baseline
Exit code: 1
Input supplied (JSON escaped): "\nhovercraft\nx\n"
```

```
==== Running minutes_parked ====
Enter entry and exit time (e.g. 09:30-11:45): Traceback (most recent call last):
  File "/home/d3va/Downloads/Academics_General/Sem-5/SE/lab7/SELABS/Lab-
7/baseline/parking_fee.py", line 33, in <module>
    main()
    ~~~~^^
  File "/home/d3va/Downloads/Academics_General/Sem-5/SE/lab7/SELABS/Lab-
7/baseline/parking_fee.py", line 19, in main
    result = minutes_parked(user_input)
  File "/home/d3va/Downloads/Academics_General/Sem-5/SE/lab7/SELABS/Lab-
7/baseline/parking_fee.py", line 4, in minutes_parked
    entry, exit_time = times.split('-')
    ^^^^^^^^^^^^^^^^
ValueError: not enough values to unpack (expected 2, got 1)
```

Name: Devansh Gupta	SRN: PES1UG24CS145	Section: C

Section 3

Section 3 - Steps 3 and 4: Fuzz tests expose failures

Each starter property uses @given(st.text() | st.none()) with max_examples=500 and deterministic generation. It checks that the function does not throw and that any returned value has a valid basic shape.

All three tests fail against the original code. Hypothesis reports seven distinct failing outcomes across those three tests: time None and empty text; rate None and empty text; plate None, empty text, and '00'.

The full tracebacks and minimized inputs are preserved in reports/before/pytest.txt. The screenshot shows a separate run with traceback display disabled for readability; no failures are hidden from the test result.

Screenshot 3: Three failing Hypothesis properties

Lab 7 · Before · pytest_summary

Browser screenshot of saved command output. Output below is reproduced verbatim.

```
$ python -m pytest --import-mode=importlib fuzz_snapshot.py::test_minutes_parked_no_crash
fuzz_snapshot.py::test_hourly_rate_no_crash fuzz_snapshot.py::test_format_plate_no_crash -q --
tb=no --color=no
Working directory: Lab-7/baseline
Exit code: 1
```

```
FFF                                                                      [100%]
========================== short test summary info ===========================
FAILED fuzz_snapshot.py::test_minutes_parked_no_crash - ExceptionGroup: Hypot...
FAILED fuzz_snapshot.py::test_hourly_rate_no_crash - ExceptionGroup: Hypothes...
FAILED fuzz_snapshot.py::test_format_plate_no_crash - ExceptionGroup: Hypothe...
3 failed in 0.44s
```

Section 4

Section 4 - Bug analysis and input policy

minutes_parked: None raises AttributeError; empty text raises ValueError. Directed checks also show that 25:00-26:00 returns 60, 09:60-10:00 returns 0, and 11:00-10:00 returns -60.

hourly_rate: None raises AttributeError and unknown or empty text raises KeyError.

format_plate: None raises AttributeError, empty text raises IndexError, and '00' produces two digits followed by three spaces. A long plate such as KA01AB123456 is silently truncated.

Contract: Invalid inputs return None. Time fields must use ASCII HH:MM-HH:MM, valid clock ranges, and same-day forward order. Equal times return 0; overnight stays need a date and are rejected here.

Rates retain bike=10, car=30, truck=60, with surrounding whitespace and case normalization. Plates accept the lab's AA99AA9999 syntax in compact or single-space grouped form and produce uppercase AA 99 AA 9999.

Name: Devansh Gupta	SRN: PES1UG24CS145	Section: C

Section 5

Section 5 - Step 5: Fix and rerun the application

The helpers check types before using string methods. Complete regex matches prevent partial parsing; explicit clock checks reject invalid values; safe dictionary lookup handles unknown vehicles; plate validation precedes formatting.

The CLI distinguishes None from a valid zero and prints a useful error for each invalid entry, continuing to the remaining prompts. EOF and Ctrl+C are handled without tracebacks.

The normal example retains its original outputs. Invalid input now produces messages for all three helpers and exits successfully.

Screenshot 4: Fixed application, valid input

Lab 7 · After · application_valid

Browser screenshot of saved command output. Output below is reproduced verbatim.

```
$ python -u parking_fee.py
Working directory: Lab-7/.
Exit code: 0
Input supplied (JSON escaped): "09:30-11:45\ncar\nka01ab1234\n"
```

```
==== Running minutes_parked ====
Enter entry and exit time (e.g. 09:30-11:45): Minutes Parked: 135

==== Running hourly_rate ====
Enter vehicle type (bike/car/truck): Hourly Rate: 30

==== Running format_plate ====
Enter a number plate (e.g. ka01ab1234): Formatted Plate: KA 01 AB 1234
```

Screenshot 5: Fixed application, invalid input

Lab 7 · After · application_invalid

Browser screenshot of saved command output. Output below is reproduced verbatim.

```
$ python -u parking_fee.py
Working directory: Lab-7/.
Exit code: 0
Input supplied (JSON escaped): "\nhovercraft\nx\n"
```

```
==== Running minutes_parked ====
Enter entry and exit time (e.g. 09:30-11:45): Invalid time range. Use HH:MM-HH:MM in
same-day order.

==== Running hourly_rate ====
Enter vehicle type (bike/car/truck): Invalid vehicle type. Choose bike, car, or
truck.

==== Running format_plate ====
Enter a number plate (e.g. ka01ab1234): Invalid number plate. Use KA01AB1234 or KA
01 AB 1234 format.
```

Name: Devansh Gupta	SRN: PES1UG24CS145	Section: C

Section 6

Section 6 - Step 6: Rerun the tests

The same three starter properties now pass. The expanded suite also checks exact valid results, time boundaries and ordering, case/whitespace normalization, plate idempotence, malformed formats, unsupported Python types, and command-line behavior.

Generated valid examples use independent expected values, including datetime arithmetic for durations and explicit known rate values. Returning None for every input would fail these properties.

Final pytest result: 88 passed in 4.68s. Raw results are saved in reports/after/. The screenshot below shows the complete test suite, including CLI regression tests.

Screenshot 6: Final complete test suite

Lab 7 · After · all_tests

Browser screenshot of saved command output. Output below is reproduced verbatim.

```
$ python -m pytest -q --color=no
Working directory: Lab-7/.
Exit code: 0
```

```
.................................................................... [ 81%]
................                                                     [100%]
88 passed in 4.68s
```

Section 7

Section 7 - Deliverables and evidence

PDF: PES1UG24CS145_SE_Lab7.pdf. Fixed application: parking_fee.py. Completed Hypothesis suite: test_parking_fee.py. Additional CLI tests: test_cli.py.

Supporting files: INPUT_CONTRACT.md, BUG_ANALYSIS.md, unchanged baseline, raw reports, screenshot PNGs, and tools to reproduce the evidence and this PDF.

Screenshots are browser captures of saved command output reproduced verbatim. The command, supplied input, working directory, and exit code appear above the output. Manifests record UTC timestamps and SHA-256 hashes for the application, tests, and output.

Fuzzing identified crashes and malformed output with small examples. Boundary checks added silent validation failures. The revised suite checks both rejection and correct handling of valid inputs; passing it does not prove correctness for all possible inputs.